

Yenepoya(Deemed to be University)
Project – High Level Design
on
Zero Day Scanner for E-commerce Web Portals

Student Name: Punnya K P

Industry Mentor:
Shashank koram

Guided by: Suzana Sharol
Prabhakar

Table of Contents

1. Introduction.....	6
1.1. Scope of the document.....	6
1.2. Intended Audience.....	6
1.3. System overview.....	6
2. System Design.....	6
2.1. Application Design.....	6
2.2. Process Flow.....	6
2.3. Information Flow.....	7
2.4. Components Design.....	8
2.5. Key Design Considerations.....	8
2.6. API Catalogue.....	8
3. Data Design.....	8
3.1. Data Model.....	8
3.2. Data Access Mechanism.....	9
3.3. Data Retention Policies.....	9
3.4. Data Migration.....	9
4. Interfaces.....	9
5. State and Session Management.....	10
6. Caching.....	10
7. Non-Functional Requirements.....	10
7.1. Security Aspects.....	10
7.2. Performance Aspects.....	10
8. References.....	11

1. INTRODUCTION

1.1 Scope of the Document

This document presents the **High-Level Design (HLD)** of the *Zero-Day Scanner for E-commerce Web Portals*. It describes:

- Overall system architecture
- Component interactions
- Data flow and processing pipeline
- API structure
- Design decisions and constraints

The document acts as a **blueprint** for implementation and evaluation.

1.2 Intended Audience

- Academic evaluators and project reviewers
- Developers and contributors
- Cybersecurity learners and researchers
- QA engineers and SEO analysts

1.3 System Overview

The Zero-Day Scanner is a web-based and CLI-based security analysis tool designed to scan e-commerce websites and detect:

- Broken links (404, 5xx)
- Redirect chains (3xx)
- Soft-404 pages
- Hidden/unmonitored subdomains

- Forms and assets for security inspection

It automates **reconnaissance + crawling + validation**, which is otherwise manual and inefficient.

2. SYSTEM DESIGN

2.1 Application Design

The system follows a client-server architecture where the frontend interacts with a Flask-based backend. The backend controls the scanning process and communicates results back to the user.

The frontend is responsible for user interaction, while the backend handles all heavy processing tasks like crawling and scanning. A separate scanning engine manages the core logic in a modular way.

Key parts of the application include:

- Frontend (UI Layer):
Handles user input, displays logs, and shows results in tables and dashboards
- Backend (Flask Server):
Manages API requests, scan execution, and data handling
- Core Scanner Engine:
Performs subdomain enumeration, crawling, and analysis

2.2 Process Flow

The execution of the system follows a clear step-by-step pipeline. When the user starts a scan, the request is processed asynchronously so that the UI remains responsive.

Initially, the system accepts the target URL and creates a unique scan session. A background thread is then started to perform scanning operations. The scanner first identifies subdomains, then crawls links within the domain up to a defined depth.

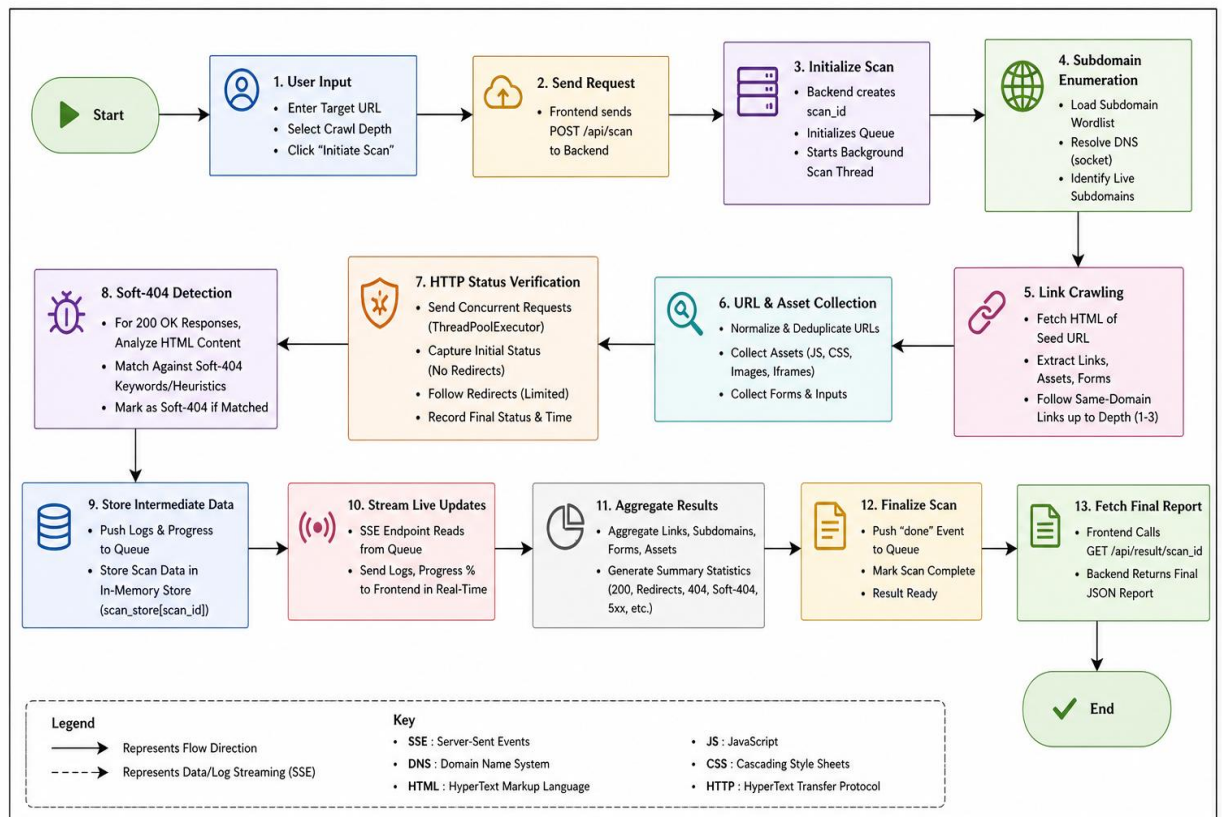
During crawling, the system extracts links, assets, and forms. These are then verified using multiple parallel HTTP requests. At the same time, special checks are performed to detect soft-404 pages and redirect chains.

Finally, the collected data is aggregated into a structured report and sent back to the frontend for display.

Important stages include:

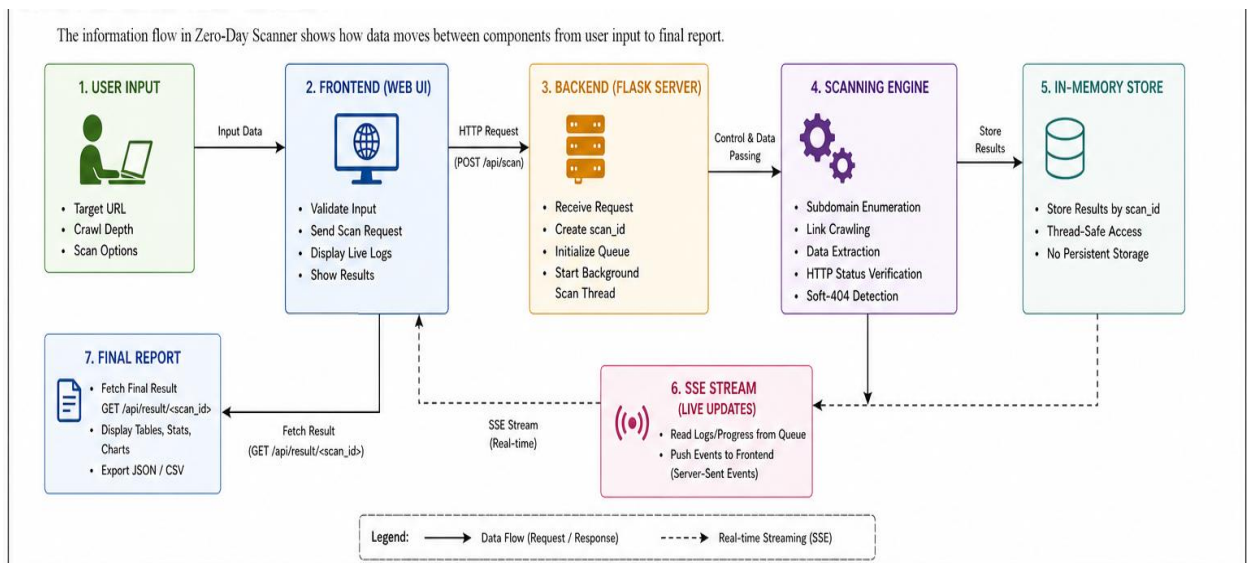
- User input and scan initiation
- Background scanning process
- Parallel HTTP status checking
- Data aggregation and reporting

The overall process flow of the Zero-Day Scanner is shown below.



2.3 Information Flow

- The system processes data in a pipeline format where raw input is transformed into structured output.
- The flow begins with the user-provided URL, which is normalized and passed into the scanning engine. The crawler extracts relevant data, and the scanner evaluates each resource using HTTP requests.
- The processed results are then stored temporarily in memory and streamed to the frontend using Server-Sent Events (SSE). Once the scan is complete, a final report is generated and displayed.
- This ensures both real-time feedback and final summarized output.



2.4 Component Design

The system is divided into multiple components to ensure modularity and clarity.

The frontend module provides an interactive interface where users can initiate scans and view results. It also includes a live terminal that displays logs in real time.

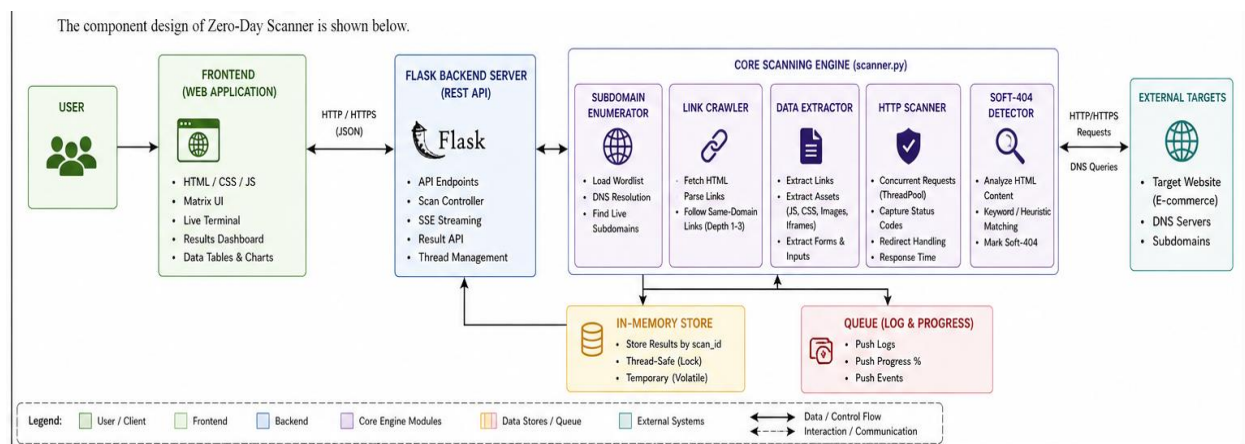
The backend module acts as the controller, handling API requests and managing the scanning workflow. It ensures proper coordination between different parts of the system.

The scanner engine is the core component responsible for performing all scanning activities. It includes submodules for crawling, HTTP verification, and data extraction.

Major components include:

- Scan Controller: manages scan lifecycle and threads
- Subdomain Module: discovers hidden domains using DNS resolution
- Crawler Module: extracts links and navigates pages
- HTTP Scanner: verifies status codes and responses
- Soft-404 Detector: identifies invalid pages with valid responses
- Result Aggregator: compiles final structured output

This modular design makes the system easier to maintain and extend.



2.5 Key Design Considerations

The design of the Zero-Day Scanner focuses on achieving high performance, simplicity, and reliable scanning of dynamic e-commerce websites. Since the system deals with network-heavy

operations like crawling and HTTP requests, special attention is given to efficiency, concurrency, and safe execution. The architecture is kept lightweight by avoiding complex dependencies such as databases, while still ensuring accurate and real-time results through optimized processing techniques.

Key considerations include:

- **Multithreading:** Uses `ThreadPoolExecutor` to perform multiple HTTP requests in parallel for faster scanning.
- **Depth Limitation:** Crawl depth is restricted (1–3 levels) to control resource usage and avoid excessive load.
- **No Persistent Storage:** Uses in-memory storage to keep the system lightweight and fast.
- **Real-Time Updates:** Implements SSE (Server-Sent Events) for live logs and progress tracking.
- **URL Deduplication:** Uses sets to avoid scanning duplicate links.
- **Redirect Handling:** Captures initial status codes and limits redirect chains to prevent loops.
- **Timeout Management:** Applies strict timeouts to prevent delays from slow or unresponsive servers.
- **Controlled Concurrency:** Limits thread count to avoid overwhelming target servers.

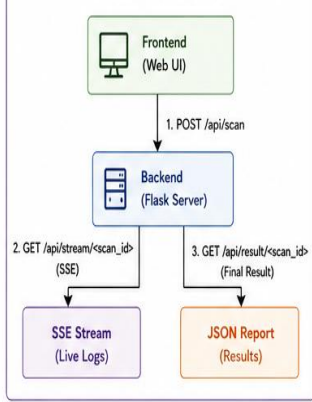
2.6 API Catalogue

API ID	METHOD	ENDPOINT	DESCRIPTION	REQUEST PAYLOAD	RESPONSE	AUTHENTICATION
API-01	POST	/api/scan	Initiates a new scan for the given target URL and crawl depth. Returns a unique scan_id.	<pre>{ "target": "https://example.com", "depth": 2 }</pre>	Success (200 OK) <pre>{ "status": "started", "scan_id": "a7f3b2c8-4d9e-11ef-9c0a-0242ac120002" }</pre>	None
API-02	GET	/api/stream/<scan_id>	Streams real-time scan logs and progress updates using Server-Sent Events (SSE).	Path Parameter: scan_id (string)	SSE Stream (text/event-stream) • log: log message • progress: percentage • status: running / done / error	None
API-03	GET	/api/result/<scan_id>	Returns the final aggregated scan result in JSON format after scan is completed.	Path Parameter: scan_id (string)	Success (200 OK) JSON Report (see Data Model)	None
API-04	GET	/api/health	Health check endpoint to verify server is running.	-	Success (200 OK) { "status": "ok" }	None

 All APIs return standard HTTP status codes.

■ Scan Management
 ■ Streaming
 ■ Results
 ■ Utility

API INTERACTION FLOW

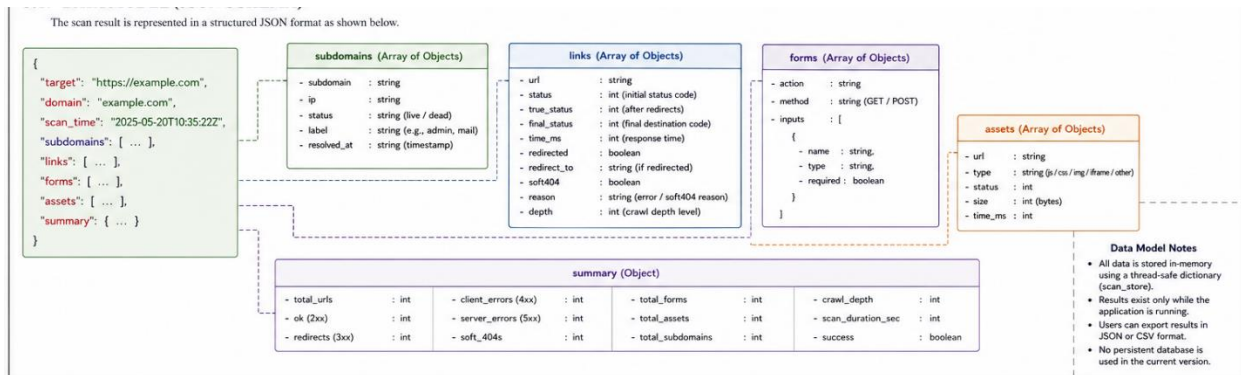


```

    graph TD
      Frontend[Frontend (Web UI)] -- "1. POST /api/scan" --> Backend[Backend (Flask Server)]
      Backend -- "2. GET /api/stream/<scan_id> (SSE)" --> SSE[SSE Stream (Live Logs)]
      Backend -- "3. GET /api/result/<scan_id> (Final Result)" --> JSON[JSON Report (Results)]
  
```

3.DATA DESIGN

3.1 Data Model



3.2 Data Access Mechanism

The frontend communicates with the backend using REST APIs. The backend interacts with external websites using HTTP requests.

Additionally, Server-Sent Events (SSE) are used to stream logs and progress updates in real time, improving user experience.

3.3 Data Retention Policy

The system does not store any data permanently. All scan results are stored temporarily in memory and are cleared once the application stops.

This ensures:

- Faster performance
- No storage overhead
- Better privacy

3.4 Data Migration

Since no database is used, data migration is not required. However, future versions may include persistent storage using databases like PostgreSQL or Redis.

4. INTERFACES

The system provides both web-based and command-line interfaces.

The web interface allows users to interact visually with dashboards and reports. The CLI version provides flexibility for advanced users.

The system also interacts externally with:

- Target websites (via HTTP/HTTPS)
- DNS servers for subdomain resolution

4. INTERFACES

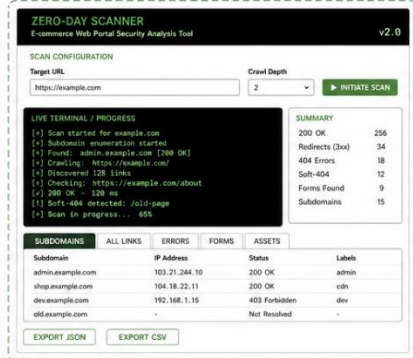
Zero-Day Scanner interacts with users and external systems through well-defined interfaces. The interfaces are categorized as User Interfaces, External Interfaces and Internal Interfaces.

4.1. USER INTERFACES

The system provides two user interaction modes.

A. WEB INTERFACE

Primary interface for most users. Built with HTML, CSS and JavaScript (Vanilla).



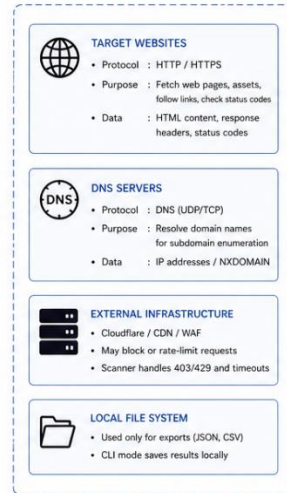
B. CLI INTERFACE

Command Line Interface for advanced users and automation.



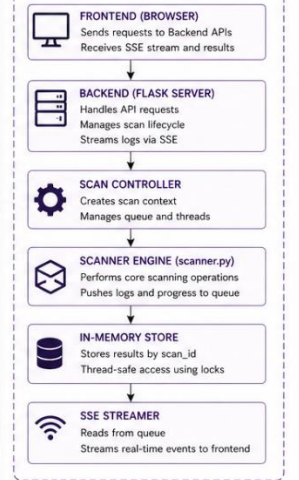
4.2. EXTERNAL INTERFACES

The system communicates with external systems over standard protocols.



4.3. INTERNAL INTERFACES

Communication between internal components within the system.



LEGEND: User Interfaces External Interfaces Internal Interfaces Interface Boundary

↔ Two-way Interaction → One-way Data Flow

5. STATE & SESSION MANAGEMENT

- The system follows a stateless architecture, where each scan is handled independently without maintaining user sessions.
- A unique scan_id is generated for every scan to track its progress and results.
- Scan data is stored temporarily in an in-memory structure (scan_store).
- The frontend uses the scan_id to receive updates and fetch final results.
- A queue mechanism is used to send real-time logs and progress updates.
- Multiple scans run in parallel with separate threads and isolated data.
- All state data is temporary and cleared after server restart.

6. CACHING

- The system currently does not use any explicit caching mechanism to ensure all scan results are fresh and up-to-date.
- Each scan request is processed independently, and data is fetched directly from target websites in real time.
- The backend (Flask) does not store or reuse previous responses, avoiding stale or outdated data issues.
- The requests library does not cache responses by default, ensuring accurate HTTP status detection.
- The browser may cache static assets like CSS and JavaScript for faster UI loading.
- DNS resolution may benefit from OS-level caching, improving subdomain lookup performance.
- Future improvements can include short-term caching (e.g., Redis or TTL cache) to reduce repeated scans and improve efficiency.

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Security Aspects

- The system includes basic security practices such as input validation and controlled request handling. It does not store any sensitive user data.
- Care is taken to avoid infinite loops and handle errors properly during scanning.

7.2 Performance Aspects

- Performance is optimized using multithreading and efficient network handling. Since most operations are I/O-bound, the system achieves good speed even with multiple requests.
- Typical scan time depends on the website size but generally ranges between a few seconds to a minute.

8. References

- Flask Documentation
- Python Requests Library
- HTTP Status Code Standards